

Matrix Multiplication and Tensor Rank Decomposition

William Van Uiter

August 2026

Table of Contents

1. The Naïve MM Algorithm	4
2. Relating Tensor Rank to Algorithmic Complexity	8
2.1. Rank and Border Rank	8
2.2. Lagrange Interpolation	8
2.3. Effect of Tensor Operation on Rank	10
2.4. Bini's Theorem	11
2.5. Non-Square Matrices Rank	14
3. Constructing an Algorithm from Rank Decomposition	15
3.1. Kronecker Delta	15
3.2. Basis Vectors	16
3.3. General MM Algorithm Construction	18
3.4. Strassen's MM Algorithm	20
3.5. Brent's Equations	23

Knowledge of what tensors fundamentally are is needed to understand tensor rank – which directly integrates into the complexity of multiplying matrices together. Common definitions for a tensor include statements like: "a tensor is an element of a tensor product" or "a tensor is an abstract mathematical object that behaves like a tensor." Although these statements are not wrong, they are tautologies, defining a tensor with tensor-specific behavior or knowledge.

To begin, a tensor is an abstract mathematical object that is invariant under transformation. An important note for tensor invariance is that the way a tensor is described, its components, may change; however, the absolute mathematical object that is being described (the tensor) will not change. This is similar to a

vector, which could be described in both polar and rectangular coordinates with different values without the physical vector direction or magnitude changing.

Addressing the first tautologous definition, imagine two separate vector spaces: V, W . Let a larger new space $Z = V \times W$. Z is a direct sum that has a basis vectors for every combination of v vectors and w vectors in V and W .

$$\forall(\vec{v}, \vec{w}) \in V \times W, \exists\{e_v, e_w\} \in Z \quad (1)$$

The problem with Z is that the axioms of linear algebra (associativity, distributivity, etc.) do not apply, so a subspace of the too large Z needs to be defined where these equations apply. Associativity and distributivity need to be defined from both the left and right with vector elements $\vec{v} \in V, \vec{w} \in W$, as well as constant c :

$$\text{Rule 1: } (c\vec{v}) \times \vec{w} = c(\vec{v} \times \vec{w}) \quad (2)$$

$$\text{Rule 2: } \vec{v} \times (c\vec{w}) = c(\vec{v} \times \vec{w}) \quad (3)$$

$$\text{Rule 3: } (\vec{v}_1 + \vec{v}_2) \times \vec{w} = \vec{v}_1 \vec{w} + \vec{v}_2 \vec{w} \quad (4)$$

$$\text{Rule 4: } \vec{v} \times (\vec{w}_1 + \vec{w}_2) = \vec{v} \vec{w}_1 + \vec{v} \vec{w}_2 \quad (5)$$

To define our subspace we will create a span, which is a set of linear combinations that obey defining rules. However, spans take in expressions not equations, so the rules must be turned into single sided expressions.

$$(1) : (c\vec{v}) \times \vec{w} - c(\vec{v} \times \vec{w}) \quad (6)$$

$$(2) : \vec{v} \times (c\vec{w}) - c(\vec{v} \times \vec{w}) \quad (7)$$

$$(3) : (\vec{v}_1 + \vec{v}_2) \times \vec{w} - \vec{v}_1 \vec{w} + \vec{v}_2 \vec{w} \quad (8)$$

$$(4) : \vec{v} \times (\vec{w}_1 + \vec{w}_2) - \vec{v} \vec{w}_1 + \vec{v} \vec{w}_2 \quad (9)$$

$$I = \text{span}\{(1), (2), (3), (4)\} \quad (10)$$

With I defined as the span of the space that follows rules 1, 2, 3, and 4, a tensor product can be defined.

$$V \otimes W = \frac{V \times W}{I} \quad (11)$$

In other words, the vector product is the quotient space of the direct sum $V \times W$ modulus I . Anything inside the space I evaluates to 0 and the tensor product can be defined as:

$$\vec{v} \otimes \vec{w} = \vec{v} \times \vec{w} + I \quad (12)$$

This simple equation can be used in tandem with expressions in I (that evaluate to zero) to prove that distributivity and associativity function with the tensor operation.

Now constants and addition can be slide between sides of the $\otimes$ symbol in expressions. A tensor product essentially joins two spaces together in a way that follows the axioms of linear algebra and a tensor is any object that lives inside this tensor space.

Lastly, different objects may be similar on structure and internal rules. These objects are said to be isomorphic ($\cong$) to one another, meaning that one can be mapped onto the other in a one 1:1 way. A result of isomorphisms is that the objects have the same dimension. Tensors and the spaces that they are part of play a crucial role in developing faster matrix multiplication (MM) algorithms. For the purposes in the paper, a tensor can thought of as the algorithm that describes the multiplication of two matrices together, where the rank of the tensor is the number of scalar multiplications needed for the operation.

1 The Naïve MM Algorithm

Let matrix A have N rows and M columns, put shortly: dimensions $N \times M$ such that the matrix can be described as:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1M} \\ a_{21} & a_{22} & \cdots & a_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{NM} \end{bmatrix} \quad (13)$$

Likewise, let matrix B have dimensions $M \times P$ so:

$$B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1P} \\ b_{21} & b_{22} & \cdots & b_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ b_{M1} & b_{M2} & \cdots & b_{MP} \end{bmatrix} \quad (14)$$

Since the number of columns in matrix A equals the number of rows in matrix B (both being the scalar number M), the two matrices can be multiplied together ($N \times M \cdot M \times P$):

$$C = A \cdot B = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1M} \\ a_{21} & a_{22} & \cdots & a_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{NM} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1P} \\ b_{21} & b_{22} & \cdots & b_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ b_{M1} & b_{M2} & \cdots & b_{MP} \end{bmatrix} \quad (15)$$

The complete operation of multiplying two matrices together uses both scalar multiplication and scalar addition that acts on a row of matrix A and a column of matrix B. More specifically, an element a_{i1} from matrix A is multiplied by element b_{1k} from matrix B, where $i \in \{1, \dots, N\}$ and $k \in \{1, \dots, P\}$, to form an intermediate product ψ_1 . Similarly, the multiplication of a_{i2} and b_{2k} form ψ_2 , a_{i3} and b_{3k} form ψ_3 , and so on. An element of matrix C C_{ik} is the sum of the ψ_n terms

that resulted by multiplying row elements by columns elements. More compactly stated:

$$C_{ik} = \sum_{j=1}^N A_{ij} B_{jk} \quad (16)$$

Alternatively, elements of matrix C can be expressed using the dot product operation between rows and columns (note: ':' means all elements of either the row or column depending on the index position) where

$$C_{ik} = A_{i:} \cdot B_{:k} \quad (17)$$

Matrix C is then developed by performing the dot product operation on each combination of rows in matrix A and columns in matrix B.

In a more visual way, matrix multiplication between row 1 and column 1 make the first element of matrix C (C_{11}):

$$C_{11} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1M} \\ a_{21} & a_{22} & \cdots & a_{2M} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & \cdots & a_{NM} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1P} \\ b_{21} & b_{22} & \cdots & b_{2P} \\ \vdots & \vdots & \ddots & \vdots \\ b_{M1} & b_{M2} & \cdots & b_{MP} \end{bmatrix} = (a_{11} \times b_{11}) + (a_{12} \times b_{21}) + \dots + (a_{1M} \times b_{M1}) \quad (18)$$

The chosen rows and columns can both be changed: the dot product of row 1 and column 2 make C_{12} , rows 2 and column 1 make C_{21} , etc. With each row-column combination performed, matrix C looks like:

$$C = \begin{bmatrix} c_{11} & \cdots & c_{1P} \\ \vdots & \ddots & \vdots \\ c_{N1} & \cdots & c_{NP} \end{bmatrix} = \begin{bmatrix} (a_{11} \times b_{11}) + \dots + (a_{1M} \times b_{M1}) & \cdots & (a_{11} \times b_{1P}) + \dots + (a_{1M} \times b_{MP}) \\ \vdots & \ddots & \vdots \\ (a_{N1} \times b_{11}) + \dots + (a_{NM} \times b_{M1}) & \cdots & (a_{N1} \times b_{NP}) + \dots + (a_{NM} \times b_{MP}) \end{bmatrix} \quad (19)$$

Although the Naïve MM algorithm looks complex when generalized for a $N \times M$ matrix multiplied by a $M \times P$ matrix, the fundamental sequence of operations is easy to grasp with practice on definite matrices. For example, the multiplication of two 2×2 matrices. If

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \quad (20)$$

then the elements of matrix C can be defined as follows

$$C_{11} = (a_{11} \times b_{11}) + (a_{12} \times b_{21}) \quad (21)$$

$$C_{12} = (a_{11} \times b_{12}) + (a_{12} \times b_{22}) \quad (22)$$

$$C_{21} = (a_{21} \times b_{11}) + (a_{22} \times b_{21}) \quad (23)$$

$$C_{22} = (a_{21} \times b_{12}) + (a_{22} \times b_{22}) \quad (24)$$

where matrix C is ultimately:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \quad (25)$$

For the multiplication of 2×2 matrices above, 8 scalar multiplication operations and 4 scalar addition operations were performed. To generalize the number of operations for the Naïve MM Algorithm, consider the fact that each element of

the general matrix C from equation 7 required M scalar multiplications and M - 1 scalar additions. There are $N \times P$ elements in matrix C, thus the total operations required are:

$$\text{Multiplications} : N \times P \times M \quad (26)$$

$$\text{Additions} : N \times P \times (M - 1) \quad (27)$$

In multiplication of two square matrices (both with dimensions $N \times N$)

$$\text{Multiplications} : N \times N \times N = N^3 \quad (28)$$

$$\text{Additions} : N \times N \times (N - 1) = N^3 - N^2 \quad (29)$$

Computers achieve scalar multiplication through a more costly process than scalar addition at the hardware level. The complexity of multiplying two matrices is conventionally determined by the necessary scalar multiplication of two $n \times n$ matrices because multiplication operations require the most computational effort. Thus, a Naïve MM Algorithm is said to be $O(n^3)$ or on the order of a cubic algorithm. An $O(n^3)$ algorithm quickly "blows up" to many computations as matrix size n increases: $n = 2 \rightarrow 8$ multiplications, $n = 3 \rightarrow 27$ multiplications, $n = 4 \rightarrow 64$ multiplications, etc. For decades, mathematicians thought the $O(n^3)$ Naïve algorithm was the most efficient possible way to multiply two matrices; however, this was later disproven.

2 Relating Tensor Rank to Algorithmic Complexity

2.1 Rank and Border Rank

The rank of a tensor ($R(T)$) is defined as the **absolute minimum** number of rank-one tensors (r) needed to construct a tensor T . Because rank is an absolute minimum relation, tensor ranks are often expressed as less than or equal to inequalities, allowing for the possibility that some linear combination of rank 1 tensors cancel each other out, reducing r . Conversely, border rank ($\underline{R}(T)$) is defined as the smallest number r where the tensor is a limit of a sequence of tensors with rank at most r . In other words, T_ϵ is an approximation tensor of T that is a sum of rank-one terms with coefficients that are polynomials or rational functions of ϵ . T_ϵ is a tensor such that:

$$\lim_{\epsilon \rightarrow 0} T_\epsilon = T \quad (30)$$

An example of a T_ϵ term may look like:

$$\frac{1}{\epsilon} (u_i \otimes v_i \otimes w_i) \quad (31)$$

where $\frac{1}{\epsilon}$ is the function coefficient to the rank one tensor $u_i \otimes v_i \otimes w_i$.

As $\epsilon \rightarrow 0$, $\frac{1}{\epsilon} \rightarrow \infty$. However, the terms with ϵ in them perfectly cancel each other out as $\epsilon \rightarrow 0$, leaving the desired tensor T . Due to **controlled** "blowing up", terms cancel out to make a simpler tensor T . In other words, border rank is the number of tensors needed to approximate T . Thus:

$$R(T_\epsilon) = \underline{R}(T) \leq R(T) \quad (32)$$

2.2 Lagrange Interpolation

Lagrange interpolation is a method to find a polynomial that passes through a set of given data points. Beginning with a starting data set: $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$,

a polynomial can be created by multiplying a data-dependent constant ($L_j(x)$) and individual y-values from the data set.

$$P(x) = \sum_{j=1}^n y_j L_j(x) \quad (33)$$

$$L_j(x) = \prod_{k=1, k \neq j}^n \frac{x - x_k}{x_j - x_k} = \begin{cases} 1 & \text{if } x = x_j \\ 0 & \text{if } x \in \{x_1, \dots, x_k | x \neq x_j\} \end{cases} \quad (34)$$

the equation for $L_j(x)$ is either 1 or 0 at x-values within the data set because either $x = x_k$ so that $x_k - x_k$ yields 0, or $x = x_j$ so $\frac{x_j - x_k}{x_j - x_k}$ is 1.

For an example, consider the (x,y) coordinate pair set $\{(0, 0), (1, 2), (-3, 2)\}$, then the $L_j(x)$ equations would be:

$$L_1(x) = \frac{x - x_2}{x_1 - x_2} \times \frac{x - x_3}{x_1 - x_3} = \frac{x - 1}{0 - 1} \times \frac{x + 3}{0 + 3} = \frac{1}{3}(1 - x)(x + 3) = \frac{1}{3}(3 - 2x - x^2) \quad (35)$$

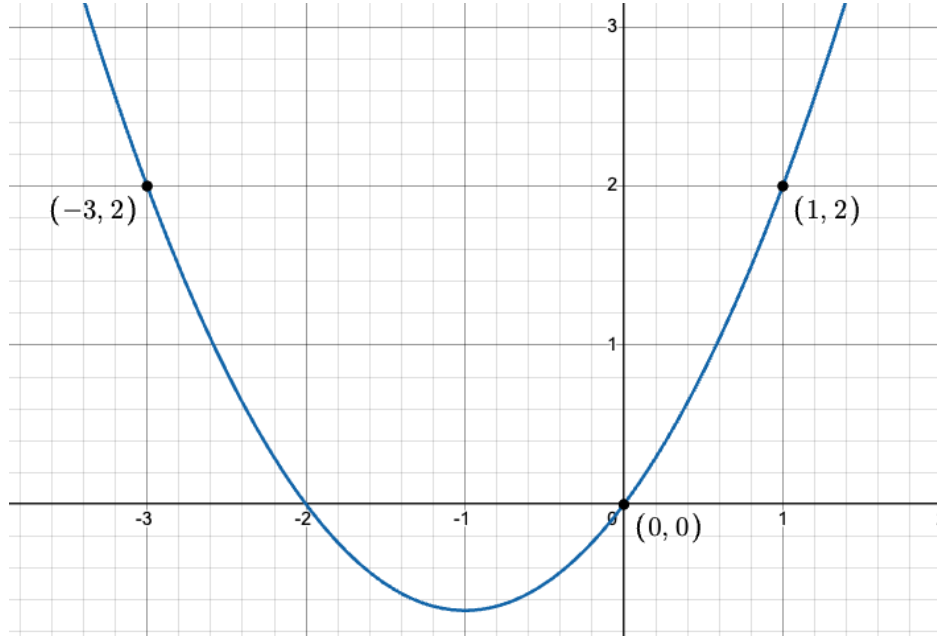
$$L_2(x) = \frac{x - x_1}{x_2 - x_1} \times \frac{x - x_3}{x_2 - x_3} = \frac{x - 0}{1 - 0} \times \frac{x + 3}{1 + 3} = \frac{1}{4}x(x + 3) = \frac{1}{4}(x^2 + 3x) \quad (36)$$

$$L_3(x) = \frac{x - x_1}{x_3 - x_1} \times \frac{x - x_2}{x_3 - x_2} = \frac{x - 0}{-3 - 0} \times \frac{x - 1}{-3 - 1} = \frac{1}{12}x(x - 1) = \frac{1}{12}(x^2 - x) \quad (37)$$

The 3 L equations can be combined to form a smooth polynomial that passes through all 3 points:

$$P(x) = y_1 L_1(x) + y_2 L_2(x) + y_3 L_3(x) = 0 + \frac{2}{4}(x^2 + 3x) + \frac{2}{12}(x^2 - x) = \frac{2}{3}(x^2 + 2x) \quad (38)$$

Thus, when $x = x_j$ (a.k.a. is at a x-coordinate of a point in the set), the corresponding L_j term dominates the equation as the other two become zero.



2.3 Effect of Tensor Operation on Rank

Define a simple tensor T as a trilinear combination with rank at most r .

$$T = \sum_{\ell=1}^r a_{\ell} \otimes b_{\ell} \otimes c_{\ell}, \quad R(T) \leq r \quad (39)$$

Another tensor T' is a copy of tensor T with similar geometric structure but with different expression or block components, an extension of tensor objects being invariant but their expressed components are not. Thus T' is defined as:

$$T' = \sum_{\ell'=1}^{r'} a'_{\ell'} \otimes b'_{\ell'} \otimes c'_{\ell'}, \quad R(T) \leq r' \quad (40)$$

Tensoring the two tensors together yields the result

$$T \otimes T' = \left(\sum_{\ell=1}^r a_{\ell} \otimes b_{\ell} \otimes c_{\ell} \right) \otimes \left(\sum_{\ell'=1}^{r'} a'_{\ell'} \otimes b'_{\ell'} \otimes c'_{\ell'} \right) = \sum_{\ell=1}^r \sum_{\ell'=1}^{r'} (a_{\ell} \otimes a'_{\ell'}) \otimes (b_{\ell} \otimes b'_{\ell'}) \otimes (c_{\ell} \otimes c'_{\ell'}) \quad (41)$$

Examining the above equation, the tensor product of T and T' is a sum of $r \times r'$ rank one tensors. Logically, the rank of the tensor product would be the product of r and r'; however, the tensor ranks are **at most** r and r' which may collapse even further. Thus it must be true that:

$$R(T \otimes T') \leq R(T)R(T') = r \times r' \quad (42)$$

Otherwise we can re-express the rank of a tensor product with itself as the rank squared, or more generally:

$$R(T^{\otimes N}) \leq R(T)^N = r^N \quad (43)$$

where $T^{\otimes N}$ means tensoring T with itself N times.

2.4 Bini's Theorem

ω is known as the matrix multiplication exponent (i.e. the minimum number of scalar multiplications required to multiply two $n \times n$ matrices together in $O(n^{\omega+\epsilon})$ time). Let ω be formally defined as:

$$\omega := \lim_{N \rightarrow \infty} \log_n (R(T^{\otimes N})) \quad (44)$$

ω does not change as N becomes larger, making ω describe complexity as a general rather than for individual matrices.

Additionally, let tensor T describe the multiplication of two $n \times n$ matrices and have max border rank r . Tensor T can be approximated with tensor T_ϵ :

$$T = \langle n, n, n \rangle = \sum_{i=1}^r u_i \otimes v_i \otimes w_i, \quad \underline{R}(T) \leq r \quad (45)$$

$$T_\epsilon = \sum_{i=1}^r u_i(\epsilon) \otimes v_i(\epsilon) \otimes w_i(\epsilon) \quad (46)$$

where u_i, v_i, w_i are polynomial terms in ϵ with a maximum degree of d . Because T_ϵ collapses into T as $\epsilon \rightarrow 0$, $\epsilon = 0$ can not be substituted into the current expression of T_ϵ . However, the definition of T_ϵ may be re-expressed to allow this. Around $\epsilon = 0$, the Taylor Series of T_ϵ expands to

$$T_\epsilon = T + \epsilon S_1 + \epsilon^2 S_2 + \cdots + \epsilon^{3d} S_{3d} \quad (47)$$

where the series is expanded to $3d$ because the tensor is a trilinear combination with terms of max degree d and S_k are a linear combination of matrix elements being multiplied together (i.e. a sub-tensor). In the above equation, it can be easily shown that when ϵ equals 0, only term T remains so that $T_\epsilon = T$.

The new goal is to separate the desired tensor T from error tensors S_k . If T_ϵ is tensored with itself N , many more combinatory terms are created (similar to polynomial multiplication); however there is a distinct $T^{\otimes N}$ term that can be brought to the front.

$$(T_\epsilon)^{\otimes N} = (T + \epsilon S_1 + \epsilon^2 S_2 + \cdots + \epsilon^{3d} S_{3d})^{\otimes N} = T^{\otimes N} + \epsilon S'_1 + \epsilon^2 S'_2 + \cdots + \epsilon^{3dN} S'_{3dN} \quad (48)$$

For simplicity, let $M = 3dN$ so that

$$(T_\epsilon)^{\otimes N} = T^{\otimes N} + \epsilon S'_1 + \epsilon^2 S'_2 + \dots + \epsilon^M S'_M \quad (49)$$

Lagrange interpolation for tensors, where each data point on a grid is a tensor (i.e. $\{T_1, T_2, \dots, T_j\}$), scales the tensor coefficients directly. To achieve the desired scaling, first, $M+1$ points must be chosen ($\{\epsilon_0, \epsilon_1, \dots, \epsilon_M\}$) and evaluated (plugged in for ϵ): $\{(T_{\epsilon_0})^{\otimes N}, (T_{\epsilon_1})^{\otimes N}, \dots, (T_{\epsilon_M})^{\otimes N}\}$. Using Lagrange interpolation, T_ϵ can be redefined as a "y-coordinate" T_{ϵ_j} multiplied by a Lagrange coefficient (λ_j).

$$(T_\epsilon)^{\otimes N} = T^{\otimes N} + \epsilon S'_1 + \dots + \epsilon^{3dN} S'_{3dN} = \lambda_0 (T_{\epsilon_0})^{\otimes N} + \lambda_1 (T_{\epsilon_1})^{\otimes N} + \dots + \lambda_M (T_{\epsilon_M})^{\otimes N} = \sum_{j=0}^M (T_{\epsilon_j})^{\otimes N} \quad (50)$$

Now letting $\epsilon \rightarrow 0$, all of the terms with ϵ evaluate to zero and $(T_\epsilon) = T$ so:

$$(T_\epsilon)^{\otimes N} = T^{\otimes N} = \sum_{j=0}^M (T_{\epsilon_j})^{\otimes N} \quad (51)$$

Taking the rank of both sides and moving the summation to the outside of the expression:

$$R(T^{\otimes N}) \leq R\left(\sum_{j=0}^M (T_{\epsilon_j})^{\otimes N}\right) = \sum_{j=0}^M R((T_{\epsilon_j})^{\otimes N}) \quad (52)$$

Recalling how tensor products affect rank, it can be determined that the rank of each of the $M+1$ $(T_{\epsilon_j})^{\otimes N}$ terms is less than or equal to r^N , giving the equation:

$$R(T^{\otimes N}) \leq (M+1) \cdot r^N = (3dN+1) \cdot r^N \quad (53)$$

Using the formal definition of ω and the relation of rank to scalar multiplication, the new equation can be substituted

$$\omega \leq \lim_{N \rightarrow \infty} \log_{n^N}(R(T^{\otimes N})) = \lim_{N \rightarrow \infty} \frac{\log((3dN + 1) \cdot r^N)}{\log(n^N)} = \lim_{N \rightarrow \infty} \left[\frac{\log(3dN + 1)}{N \log(n)} + \frac{N \log(r)}{N \log(n)} \right] = \frac{\log(r)}{\log(n)} \quad (54)$$

Thus the complexity of a two $n \times n$ matrices is $\log_n r$. This provides the framework for correlating rank decomposition to time complexity of square matrix multiplication through a simple equation.

2.5 Non-Square Matrices and Rank

A tensor's rank can be determined by its vector and covector parameters. Therefore, tensor ranks are cyclic with respect to index position because differently ordered parameters make tensors with the similar rank and size within the same ambient space, as such

$$R(< m, s, t >) = R(< s, t, m >) = R(< t, m, s >) \quad (55)$$

$$< m, s, t > \otimes < s, t, m > \otimes < t, m, s > \cong < mst, mst, mst > \quad (56)$$

$$R(< mst, mst, mst >) \leq r \cdot r \cdot r = r^3 \quad (57)$$

which relies on what was proven in equation 43. By extension, tensoring to the N_{th} power:

$$N := (mst)^n \quad (58)$$

$$R(T^{\otimes n}) = R(< (mst)^n, (mst)^n, (mst)^n >) = R(< N, N, N >) \leq (r^3)^n = r^{3n} \quad (59)$$

because N by definition relates to the sides of the matrices, it can be said that

$$N^\omega = (mst)^{\omega n} \leq r^{3n} \quad (60)$$

$$\log_{mst}(mst)^{\omega n} = \omega \log_{mst}(mst) \leq \log_{mst}(r^{3n}) = 3n \log_{mst}(r) \quad (61)$$

$$\omega \leq 3 \log_{mst}(r) \quad (62)$$

It is easily verifiable that this is the same equation as the one derived during Bini's Theorem. If $m=s=t=n$ the 3 coefficient cancels with the n^3 term to get the result of equation 54.

3 Constructing an Algorithm from Rank Decomposition

3.1 Kronecker Delta

For 2 dimensional matrix multiplication, the function of the Kronecker Delta (δ) is very similar to an identity matrix (I). Elements in the identity matrix are either 1 or 0, with 1s being arranged only on the diagonal of the matrix. For example, the identity matrix of matrix rank 3 would be:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (63)$$

and a more general M rank identity matrix would look like:

$$I_M = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix} \quad (64)$$

Identity matrices have a characteristic identity, such that elements within the matrix are 1 if and only if the row index is equal to the column index: $I_{ij} = 1 \iff i = j$. Otherwise, the element is zero.

The Kronecker Delta relies on the same concept except is a function that can exist in higher dimensions than two. Therefore:

$$\delta_{ij} = I_{ij} \quad (65)$$

$$\delta_{ij} = 1 \iff i = j, \quad \delta_{ij} = 0 \iff i \neq j \quad (66)$$

The Kronecker delta is used to pick out terms where indices match and also to show that states or vectors are orthogonal (i.e. independent) to one another.

3.2 Basis Vectors

Basis vectors are a set of vectors that describe a coordinate system. The vectors are written so that a vector describes a single unit in the adopted system along the span of the vector, which can be thought of as a coordinate axis. The components of the basis vectors are often written in a way that would describe how they exist within a traditional cartesian coordinate system.

For a basic 3 dimensional cartesian coordinate space ($\mathbb{R}^3$), there are 3 basis vectors: $\hat{i}, \hat{j}, \hat{k}$.

$$\hat{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \text{x-axis} \quad (67)$$

$$\hat{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \text{y-axis} \quad (68)$$

$$\hat{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \text{z-axis} \quad (69)$$

Any vector $\vec{v}$ that exist within the space can be expressed as a linear combination of basis vectors: $\vec{v} = a\hat{i} + b\hat{j} + c\hat{k}$.

In an n dimensional cartesian coordinate space, n basis vectors would be needed to form a full coordinate system to describe the space. A general basis vector $e_i \in \{e_1, e_2, \dots, e_n\}$ would have n elements in a column. In a rectangular coordinate system, each of these elements would be zero with the exception of the i_{th} element which would be 1.

$$e_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad (70)$$

In non-rectangular systems not all of the elements are 1 or 0. For example, an xy-plane could be described in polar coordinates with basis vectors

$$\hat{r} = \cos(\theta)\hat{i} + \sin(\theta)\hat{j} = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix} \quad (71)$$

$$\hat{\theta} = -\sin(\theta)\hat{i} + \cos(\theta)\hat{j} = \begin{bmatrix} -\sin(\theta) \\ \cos(\theta) \end{bmatrix} \quad (72)$$

3.3 General MM Algorithm Construction

Beginning with matrices A ($M \times N$) and B ($N \times P$), A and B can be multiplied together to yield matrix C of dimension $M \times P$. The matrices all must first be flattened into column vectors in a traditional column-major order.

Column-major vectorization of matrices takes each column of a matrix and stacks one column atop the other. For example, if matrix A was vectorized:

$$vec(A) = \sum_{i=1}^N (e_i \otimes I_M) A e_i = vec\left(\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1N} \\ a_{21} & a_{22} & \cdots & a_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M1} & a_{M2} & \cdots & a_{MN} \end{bmatrix} \right) = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{M1} \\ a_{12} \\ a_{22} \\ \vdots \\ a_{M2} \\ \vdots \\ a_{1N} \\ a_{2N} \\ \vdots \\ a_{MN} \end{bmatrix} \quad (73)$$

where e_i is the i_{th} base vector that has a 1 in the i_{th} row and 0 in every other row. Additionally, I_M is the identity matrix of dimension M which has 1s only on its diagonal.

Matrix A gets flattened into vector $\vec{a}$ with $M \times N$ rows, matrix B gets flattened into $\vec{b}$ with $N \times P$ rows, and matrix C gets flattened into $\vec{c}$ with $M \times P$ rows. Additionally, define tensor T of size $(MN) \times (NP) \times (MP)$ and rank r. Tensor T exist as a rank r trilinear product of spaces U (size $M \times N \times r$), V (size $N \times P \times r$), and W (size $M \times P \times r$). The MM multiplication algorithm can be expressed through tensor T as

$$T = \sum_{m=1}^r \vec{u}_m \otimes \vec{v}_m \otimes \vec{w}_m \quad (74)$$

The job of the tensor is to encode the rule:

$$C_{ik} = \sum_{j=1}^N A_{ij} B_{jk} \quad (75)$$

Let $\alpha = (i, j)$, $\beta = (j', k)$, and $\gamma = (i', k')$. Substituting the new indexing values into the regular equation for matrix product elements:

$$C_{\gamma} = \sum_{j=1}^N A_{\alpha} B_{\beta} \quad (76)$$

This ensures that matrix A and C have the same row, the shared index matches for A and B, and the column of matrix B matches the column of matrix C, which sets up a non-linear system of equations based on components from matrix A and others from matrix.

$$T_{\alpha\beta\gamma} = \begin{cases} 1 & i = i', j = j', k = k' \\ 0 & \text{else} \end{cases} \quad (77)$$

for each $m \in \{1, \dots, r\}$:

$$L_m = \sum_{i=1}^M \sum_{j=1}^N U_{\alpha m} A_{\alpha} \quad (78)$$

$$R_m = \sum_{j=1}^N \sum_{k=1}^P V_{\beta m} B_{\beta} \quad (79)$$

Equations L_m and R_m define the left hand element from matrix A and right hand element from matrix B that are multiplied together. The product of these components from matrices A and B can then be multiplied into an intermediate scalar product:

$$M_m = L_m \times R_m \quad (80)$$

$$C_{\gamma} = \sum_{m=1}^r W_{\gamma m} M_m \quad (81)$$

C_{γ} describes how intermediate products are combined to form the final matrix C according to the indexed element of matrix W being 1, 0, or -1.

3.4 Strassen's MM Algorithm

The rank decomposition to MM algorithm logic presented in the section can be applied to find Strassen's Algorithm. Beginning with 2×2 matrices A, B, and C defined as:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} \quad C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix} \quad (82)$$

Elements of matrices A, B, C need not be scalar values but instead can be block matrices, in which case the algorithm would be applied recursively. Next, the matrices are vectorized:

$$\vec{a} = \text{vec}(A) = \begin{bmatrix} a_{11} \\ a_{21} \\ a_{12} \\ a_{22} \end{bmatrix} = \begin{bmatrix} A_1 \\ A_2 \\ A_3 \\ A_4 \end{bmatrix} \quad \vec{b} = \text{vec}(B) = \begin{bmatrix} b_{11} \\ b_{21} \\ b_{12} \\ b_{22} \end{bmatrix} = \begin{bmatrix} B_1 \\ B_2 \\ B_3 \\ B_4 \end{bmatrix} \quad \vec{c} = \text{vec}(C) = \begin{bmatrix} c_{11} \\ c_{21} \\ c_{12} \\ c_{22} \end{bmatrix} = \begin{bmatrix} C_1 \\ C_2 \\ C_3 \\ C_4 \end{bmatrix} \quad (83)$$

Now that the matrices are flattened into column vectors $\vec{a}$, $\vec{b}$, and $\vec{c}$ a tensor T can be defined similar to equation 51.

$$T_{ijk} = \sum_{m=1}^r U_{im} V_{jm} W_{km} \quad (84)$$

U , V , and W are known as factor matrices and are the critical component that makes finding an MM algorithm hard. Equation 59 must hold true, making a large system of equations that is intractable to solve by hand but possible with machines and the use of **Brent's Equations**. Values within the matrices are either 1, 0, or -1 and the dimensions of the matrices are $r \times s$ (the side of the square matrices and the rank of the tensor). For Strassen's algorithm they are:

$$U = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{bmatrix} \quad V = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \quad W = \begin{bmatrix} 1 & 0 & 0 & 1 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (85)$$

Essentially, each row in factor matrix U corresponds to an equation with the 4 A terms, similarly for factor matrix V and B terms.

$$\vec{a}_u = U \times \vec{a} = \begin{bmatrix} (A_1 + A_4) & (A_2 + A_4) & A_1 & A_4 & (A_1 + A_3) & (A_2 - A_1) & (A_3 - A_4) \end{bmatrix} \quad (86)$$

$$\vec{b}_v = V \times \vec{b} = \begin{bmatrix} (B_1 + B_4) & B_1 & (B_3 - B_4) & (B_2 - B_1) & B_4 & (B_1 + B_3) & (B_2 + B_4) \end{bmatrix} \quad (87)$$

There are many different permutations of these matrices because rows can be switched in U, V, and W as long as they are done the same across the 3 matrices. Additionally, rows in matrices U and V can be multiplied by -1 as long as its done to corresponding rows in both U and V because $-1 \times -1 = 1$; however, this should not be also done to W because $-1 \times -1 \times -1 = -1$.

Next, vectors $\vec{a}_u$ and $\vec{b}_v$ are combined with the Hadamard Product (each element of $\vec{a}_u$ is multiplied by its corresponding element in $\vec{b}_v$).

$$\vec{H} = \vec{a}_u \circ \vec{b}_v = \begin{bmatrix} \vec{a}_{u1} \times \vec{b}_{v1} & \vec{a}_{u2} \times \vec{b}_{v2} & \cdots & \vec{a}_{u7} \times \vec{b}_{v7} \end{bmatrix} \quad (88)$$

Finally, matrix C is calculated by multiplying vector H by matrix W:

$$C = W \times \vec{H} \quad (89)$$

This algorithm has some limitations. Both the multiplier and multiplicand matrices must be square and have equal dimensions. Furthermore, this dimension must be a power of 2. This is so that matrices can be broken into smaller block matrices for the algorithm. The example above that had 4 elements in matrices is the same one used for larger size matrices only the algorithm is iterated recursively. Similar, square, power 2 matrices may be achieved by padding matrices that do

not meet these requirements with 0 elements.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \rightarrow \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & a_{23} & 0 \\ a_{31} & a_{32} & a_{33} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (90)$$

For a 2×2 matrix, Strassen's algorithm requires 7 scalar multiplications (which is the rank of the tensor describing the MM algorithm) and 18 additions. Thus, using equation 46:

$$\omega \leq 3 \log_{(2 \times 2 \times 2)} 7 = \log_2 7 \approx 2.807 \quad (91)$$

Strassen's algorithm lowered the complexity of MM from $O(n^3)$ to around $O(n^{2.807})$, greatly increasing the efficiency of MM algorithms.

3.5 Brent's Equations

Turning to a search for valid α, β, γ values, the logic must be turned into a boolean satisfiability problem that can be fed into a computer solver. Given tensor $T = \langle m, n, p \rangle$ that requires t multiplications, otherwise put as $T = \langle m, n, p; t \rangle$, t auxiliary products M_k can be created

$$\sum_{r=1}^t \alpha_{i'jr} \beta_{j'kr} \gamma_{ikr} = \delta_{ii'} \delta_{jj'} \quad \text{for all } i, i' \in [m], j, j' \in [n], k \in [p] \quad (92)$$

where α_{ijk} are coefficients of linear combinations of matrix A elements, β_{ijk} are coefficients of linear combinations of matrix B elements, and γ_{ijk} are coefficients that dictate how to combine auxiliary products.

For the multiplication of 2×2 matrices, where $m = n = p = 2$, the target tensor grid would create $(m \times n)(n \times p)(p \times n) = 2^6 = 64$ equations where each equation is either equal to 1 or zero depending on the Kronecker Delta. For example, for

Strassen's algorithm of 7 scalar multiplications:

$$\sum_{r=1}^7 \alpha_{11r} \beta_{11r} \gamma_{11r} = 1 \quad (93)$$

$$\sum_{r=1}^7 \alpha_{12r} \beta_{12r} \gamma_{12r} = 0 \quad (94)$$

4 variables per multiplication step (the number of elements in each matrix), 7 multiplication steps (tensor rank), and 3 factor matrices (α, β, γ) makes 84 unknown variables to solve for in only 64 equations. This is impossible for humans to solve by hand but can be solved in many ways with machine help. Because the numerical values of the equations are so limited: equations either equal 1 or 0 and variables are restricted to -1, 0, or 1, it is easier to map the non-continuous logic of the problem to boolean satisfiability (SAT) constraints.

The variables of α, β , and γ are translated into binary bits. Then, the multiplication of these variables is represented as the $\wedge$ symbol.

$$\text{Let } d = \alpha \wedge \beta \quad (95)$$

$$\text{Let } X = d \wedge \gamma = \alpha \wedge \beta \wedge \gamma \quad (96)$$

represents the multiplication of the three variables together using intermediate variable 'd'.

Next the phrase "d is True if and only if α AND β are True" is translated into conjunctive normal form (CNF) logical propositions:

$$(\neg d \vee \alpha) \wedge (\neg d \vee \beta) \wedge (d \vee \neg \alpha \vee \neg \beta) \quad (97)$$

a similar phrase translation is performed for variable X.

Normally, the XOR gate ($\oplus$) is the logical proposition representation of a direct sum of two variables; however, XOR is not in CNF. So instead a larger representation must be used:

$$y = a \oplus b = (\neg y \vee \neg a \vee \neg b) \wedge (\neg y \vee a \vee b) \wedge (\neg y \vee \neg a \vee b) \wedge (\neg y \vee a \vee \neg b) \quad (98)$$

To solve for the variables a large direct sum of $(X_1 \oplus X_2 \oplus \cdots \oplus X_7 = 0 \text{ or } 1)$ is required. Many intermediate variables are used:

$$y_1 = X_1 \oplus X_2, \quad y_2 = y_1 \oplus X_3, \quad \text{etc} \quad (99)$$

Finally, unit clauses (0 or 1) are programmed into the solver. If an equation is equal to 0 then $(\neg y_f)$, which says that the final y should be false. If an equation is equal to 1 then (y_f) , or the equation is true.

Additionally, the SAT solver can be primed on the fact that variable solutions must be -1, 0, or 1. This is represented using two bits, 1 values bit (V) and a sign bit (S), to represent the possible variable values:

$$0 \rightarrow V = 0, S = 0$$

$$1 \rightarrow V = 1, S = 0$$

$$-1 \rightarrow V = 1, S = 1$$

The fourth situation ($V = 0, S = 1$) would be -0, which is not possible. Therefore, the line $\neg(\neg V \wedge S)$ is added as a requirement. This system is applied to the multiplication and sum of variables:

$$V_X = V_\alpha \wedge V_\beta \wedge V_\gamma, \quad S_X = S_\alpha \oplus S_\beta \oplus S_\gamma \quad (100)$$

Using ternary full-adder circuits, the solver can carry "overflow bits" to the next column and flip a tracking bit, allowing for trickle down effects from 1 solution that speed up the search for all variables.